We also walk through the technical details of the contract testing life cycle, particularly with the CDCT approach. While reading about the life cycle, also think about the human element of each step within software development, that is, who the people involved in the process are and what type of conversations you would have at each stage. The reason for mentioning human interactions is that contract testing is a great conversation starter for teams when there is an integration problem. The results from contract tests give teams the equivalent of a stack trace between systems.

Say we have a web application, and put yourself in the shoes of a team in charge. The team experienced an integration problem in the web application. For example, a data field in the response data has gone missing. A web developer starts by digging into their code to check that they are using the correct version of the API. Then, they check how they constructed the request and that everything has stayed the same. They can't see any changes to the specific HTTP request and confirm the API version. The only option left is for them to message the team who owns the API service to understand why the field has disappeared. The API team also doesn't think they have changed that HTTP response. However, they looked through their recent commits to find a trivial change has been made where the field naming convention has been changed from camel case to snake case to align with the existing API documentation. The API developer remembers sending technology-wide emails to notify the teams of the breaking change, but the web developer must not have seen the memo.

> **NOTE** The lesson of this story is that humans will make mistakes, especially regarding communication. Any tools and processes enabling better communication can only be good.

This chapter also mentions different communication types, including API, GraphQL, and event-driven messaging. Contract testing is commonly associated with APIs but isn't applicable only to REST APIs. It also supports GraphQL-style and messages architecture. The same concepts apply to each communication type. Therefore, once you understand the core concepts described in this chapter, we suggest skipping to the chapter that covers your relevant kind of implementation.